

DEVELOPMENT PROPOSAL

Project Storefront 2.0: Next-Gen Headless E-Commerce Platform

A Technical Web Development Proposal for Composable Commerce Migration & Performance Optimization

PREPARED FOR: Chief Technology Officer, Horizon Retail Brands

PREPARED BY: Enterprise Web Engineering Division

DATE: July 10, 2026

VERSION: 2.0 (Proposal Draft)

1. EXECUTIVE SUMMARY

Horizon Retail Brands' current online marketplace relies on a legacy, monolithic infrastructure that suffers from high latency during traffic spikes, a restrictive templating engine, and an checkout bounce rate of 28%. With core web vitals consistently grading in the "Poor" category, mobile conversion rates have declined steadily over the past three quarters.

Project Storefront 2.0 outlines a complete modernization of our web ecosystem. By decoupling the frontend presentation layer from backend business logic (Headless Composable Commerce), we will introduce a lightning-fast, highly responsive user interface built on Next.js, powered by a scalable GraphQL API mesh layer and a cloud-native serverless hosting architecture.

Core Quantitative Performance Targets

- Achieve a sub-second Largest Contentful Paint (LCP) time across mobile and desktop environments.
- Drive a projected 14% uplift in checkout conversions by engineering a single-page, friction-free checkout application.
- Eliminate downtime risks during high-volume sales events (e.g., Black Friday) through edge-network page caching and serverless auto-scaling.

2. ARCHITECTURAL CHALLENGES & CORE OPPORTUNITIES

A deep audit of the existing web architecture identified three foundational performance bottlenecks:

- **Tight Coupling:** Changes to superficial UI design require re-compiling backend core code bases, slowing development deployment cycles to a bi-weekly cadence.
- **Bloated Client-Side Scripting:** Excessive legacy JavaScript plugins defer initial thread execution, leading to a poor Interaction to Next Paint (INP) score of >450ms.
- **Inflexible Content Workflows:** Marketing teams are entirely dependent on engineering tickets to execute simple product banners and landing page promotions.

3. PROPOSED TECHNOLOGY STACK & ENGINEERING STRATEGY

The proposed decoupled architecture emphasizes modern web development best practices focusing on modularity, security, and global accessibility.

3.1 Frontend Layer (Next.js & Tailwind CSS)

We will leverage Next.js utilizing React Server Components (RSC) to maximize static page pre-rendering at the edge while streaming dynamic personalization logic where necessary. Styling will be built using utility-first Tailwind CSS to guarantee highly optimized, minimal CSS bundle assets.

3.2 API Orchestration & Composable Content Layer

An enterprise GraphQL Mesh layer will unify disparate downstream services (ERP data, Inventory Managers, and Stripe Payment APIs) into a single, federated data schema. Content management will be decoupled via a headless CMS (e.g., Contentful or Sanity), allowing real-time marketing layouts without code manipulation.

4. IMPLEMENTATION TIMELINE & ENGINEERING SPRINTS

The development cycle is organized across a 14-week agile timeline, splitting milestones into logical bi-weekly sprints.

Sprint Block	Focus Area	Key Deliverables	Timeline
Sprints 1–2	API Mesh & Foundations	GraphQL gateway initialization, data schema definitions, CI/CD pipeline setups.	Weeks 1–4
Sprints 3–4	UI Components & Engine	Design system implementation, structural grid/category layout builds, Headless CMS hooks.	Weeks 5–8
Sprint 5	Cart & Secure Checkout	Stripe custom element integrations, session state engine, localized validation.	Weeks 9–10
Sprint 6	E2E Optimization	Analytics tagging, Core Web Vital testing, load stress evaluation (100k concurrent users).	Weeks 11–12
Sprint 7	Staged DNS Cutover	A/B traffic rollout, reverse-proxy setup, system documentation handover.	Weeks 13–14

5. FINANCIAL COST MODELING & BUDGET BREAKDOWN

The financial layout includes full lifecycle agile engineering resources, automated end-to-end continuous integration tool chains, and sandbox environment provisions.

Resource Group	Engineering Scope Description	Investment (USD)
Frontend Engineering	Senior React/Next.js Engineers, Mobile Web Responsive Polishing	\$85,000
Backend & API Integration	GraphQL Federations, Middleware Microservices Development, Core API Hookups	\$65,000
DevOps & Edge Architecture	Vercel/AWS Cloud Provisioning, WAF Firewalls, CDN Edge Caching Rules	\$25,000
UI/UX Design Systems	Figma-to-Code Component Standardization, Accessible Token Definitions	\$15,000
Total Development Cost		\$190,000

6. POST-LAUNCH SLAS & STRUCTURAL REDUNDANCY

To preserve performance integrity, post-launch protocols ensure consistent uptime and fast recovery capabilities:

- **Stale-While-Revalidate (SWR) Resilience:** If downstream backend inventories crash, edge nodes continue serving cached pages seamlessly to incoming buyers rather than standard error screens.
- **Instant Rollbacks:** Every Vercel deployment builds an atomic immutable URL, allowing developers to revert a broken UI live environment to a safe working state in under 5 seconds.